

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

COURSE CODE: CSA-0409

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 50

Experiment Questions:

1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.

2.Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.

3.Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.

4.Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.

5.Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

RUBRICS FOR CALCULATION:

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS:

1. Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.

PSEUDO CODE:

```
START
Read Base
Read Limit
Read Logical Address
IF Logical Address < 0 OR Logical Address >= Limit THEN
    Display "Invalid Logical Address"
ELSE
    Physical Address = Base + Logical Address
    Display "Valid Logical Address"
    Display Physical Address
END IF
STOP
```

CODE:

```
#include <stdio.h>

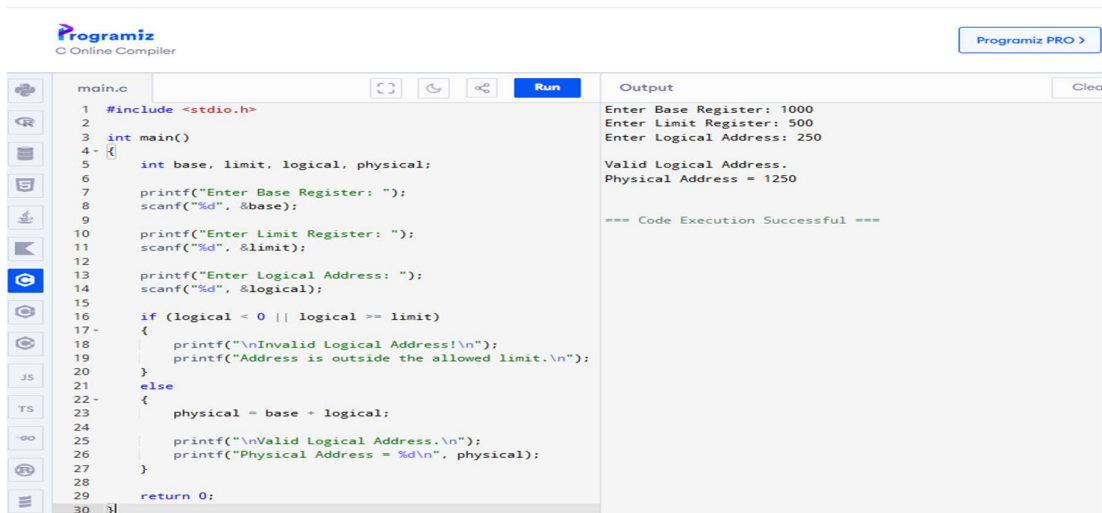
int main()
{
    int base, limit, logical, physical;
    printf("Enter Base Register: ");
    scanf("%d", &base);
    printf("Enter Limit Register: ");
    scanf("%d", &limit);
    printf("Enter Logical Address: ");
    scanf("%d", &logical);
    if (logical < 0 || logical >= limit)
```

```

    {
        printf("\nInvalid Logical Address!\n");
        printf("Address is outside the allowed limit.\n");
    }
else
{
    physical = base + logical;
    printf("\nValid Logical Address.\n");
    printf("Physical Address = %d\n", physical);
}
return 0;
}

```

VALID OUTPUT:



The screenshot shows the Programiz C Online Compiler interface. The code in main.c is as follows:

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int base, limit, logical, physical;
6
7     printf("Enter Base Register: ");
8     scanf("%d", &base);
9
10    printf("Enter Limit Register: ");
11    scanf("%d", &limit);
12
13    printf("Enter Logical Address: ");
14    scanf("%d", &logical);
15
16    if (logical < 0 || logical >= limit)
17    {
18        printf("\nInvalid Logical Address!\n");
19        printf("Address is outside the allowed limit.\n");
20    }
21    else
22    {
23        physical = base + logical;
24        printf("\nValid Logical Address.\n");
25        printf("Physical Address = %d\n", physical);
26    }
27
28    return 0;
29 }
30

```

The output window shows the following text:

```

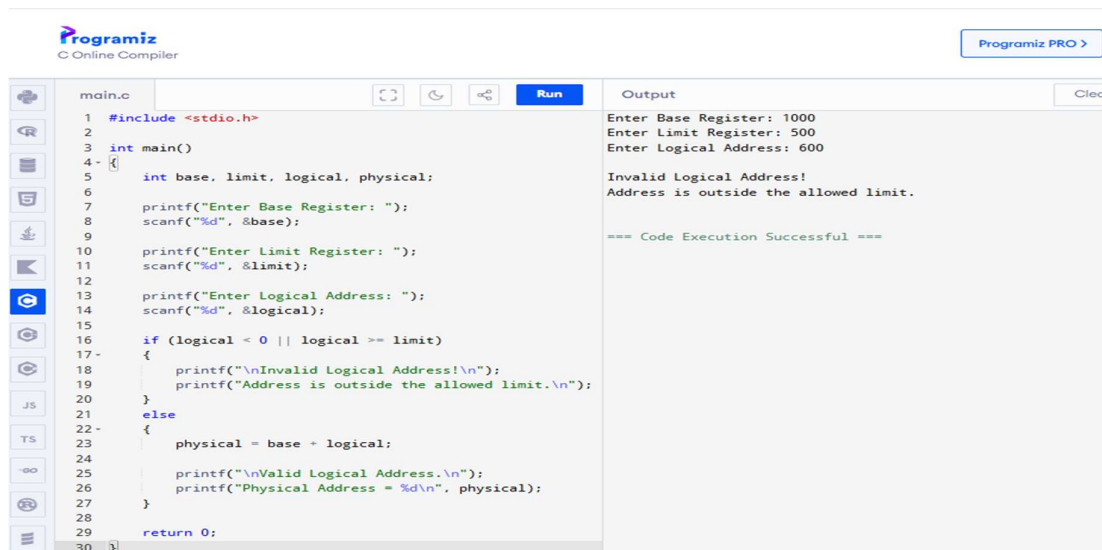
Enter Base Register: 1000
Enter Limit Register: 500
Enter Logical Address: 250

Valid Logical Address.
Physical Address = 1250

=== Code Execution Successful ===

```

INVALID INPUT:



The screenshot shows the Programiz C Online Compiler interface with the same code as the previous example. The output window shows the following text:

```

Enter Base Register: 1000
Enter Limit Register: 500
Enter Logical Address: 600

Invalid Logical Address!
Address is outside the allowed limit.

=== Code Execution Successful ===

```

2. Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.

PSEUDO CODE:

```
START
Read number of pages
Read number of frames
Read page size
For each page
    Read frame number
    Store in page table
End For
Read page number
Read offset
If page number is invalid
    Display "Invalid Page Number"
Else if offset is invalid
    Display "Invalid Offset"
Else
    Frame number = Page Table[Page Number]
    Physical Address = (Frame Number × Page Size) + Offset
    Display Physical Address
End If
STOP
```

CODE:

```
#include <stdio.h>

int main()
{
    int pages, frames, pageSize;
    int pageTable[100];
    int pageNumber, offset;
```

```

int physicalAddress;
printf("Enter number of pages: ");
scanf("%d", &pages);
printf("Enter number of frames: ");
scanf("%d", &frames);
printf("Enter page size: ");
scanf("%d", &pageSize);
printf("\nEnter Page Table:\n");
for (int i = 0; i < pages; i++) {
    printf("Page %d -> Frame: ", i);
    scanf("%d", &pageTable[i]);
}
printf("\nEnter Page Number: ");
scanf("%d", &pageNumber);
printf("Enter Offset: ");
scanf("%d", &offset);
if (pageNumber < 0 || pageNumber >= pages) {
    printf("\nInvalid Page Number!\n");
}
else if (offset < 0 || offset >= pageSize) {
    printf("\nInvalid Offset!\n");
}
else if (pageTable[pageNumber] < 0 || pageTable[pageNumber] >= frames){
    printf("\nPage is not mapped to a valid frame!\n");
} else{
    physicalAddress = pageTable[pageNumber] * pageSize + offset;
    printf("\nPage Number = %d", pageNumber);
    printf("\nOffset = %d", offset);
    printf("\nFrame Number = %d", pageTable[pageNumber]);
    printf("\nPhysical Address = %d\n", physicalAddress);
}
return 0;
}

```

OUTPUT:

main.c	Output
<pre>1 #include <stdio.h> 2 int main() 3 { 4 int pages, frames, pageSize; 5 int pageTable[100]; 6 int pageNumber, offset; 7 int physicalAddress; 8 printf("Enter number of pages: "); 9 scanf("%d", &pages); 10 printf("Enter number of frames: "); 11 scanf("%d", &frames); 12 printf("Enter page size: "); 13 scanf("%d", &pageSize); 14 printf("\nEnter Page Table:\n"); 15 for (int i = 0; i < pages; i++) { 16 printf("Page %d -> Frame: ", i); 17 scanf("%d", &pageTable[i]); 18 } 19 printf("\nEnter Page Number: "); 20 scanf("%d", &pageNumber); 21 printf("Enter Offset: "); 22 scanf("%d", &offset); 23 if (pageNumber < 0 pageNumber >= pages) { 24 printf("\nInvalid Page Number!\n"); 25 } else if (offset < 0 offset >= pageSize){ 26 printf("\nInvalid Offset!\n"); 27 } else if (pageTable[pageNumber] < 0 28 pageTable[pageNumber] >= frames) { 29 printf("\nPage is not mapped to a valid frame!\n"); 30 } else { 31 physicalAddress = pageTable[pageNumber] * pageSize 32 + offset; 33 printf("\nPage Number = %d", pageNumber); 34 printf("\nOffset = %d", offset); 35 printf("\nFrame Number = %d", 36 pageTable[pageNumber]); 37 printf("\nPhysical Address = %d\n", 38 physicalAddress); 39 } 40 return 0; 41 }</pre>	<pre>Enter number of pages: 4 Enter number of frames: 6 Enter page size: 100 Enter Page Table: Page 0 -> Frame: 3 Page 1 -> Frame: 5 Page 2 -> Frame: 1 Page 3 -> Frame: 4 Enter Page Number: 2 Enter Offset: 50 Page Number = 2 Offset = 50 Frame Number = 1 Physical Address = 150 === Code Execution Successful ===</pre>

3. Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.

PSEUDO CODE:

```
START
Read number of pages
Read page size
FOR each page
    Read frame number
    Store frame number in page table
END FOR
Display page table
Read page number
Read offset
IF page number is invalid
    Display "Invalid Page Number"
ELSE IF offset is invalid
    Display "Invalid Offset"
ELSE
    Frame number = Page Table[Page Number]
    Physical Address =
        (Frame Number × Page Size) + Offset
    Display Page Number
    Display Frame Number
    Display Offset
    Display Physical Address
END IF
STOP
```


CODE:

```
#include <stdio.h>

int main()
{
    int pageTable[100];
    int pages, pageSize;
    int pageNumber, offset;
    int frameNumber, physicalAddress;
    printf("Enter number of pages: ");
    scanf("%d", &pages);
    printf("Enter page size: ");
    scanf("%d", &pageSize);
    printf("\nEnter Page Table Entries:\n");
    for (int i = 0; i < pages; i++)
    {
        printf("Page %d -> Frame: ", i);
        scanf("%d", &pageTable[i]);
    }
    printf("\nPage Table:\n");
    printf("-----\n");
    printf("Page Number   Frame Number\n");
    printf("-----\n");
    for (int i = 0; i < pages; i++) {
        printf("    %d        %d\n", i, pageTable[i]);
    }
    printf("\nEnter Logical Address:\n");
    printf("Enter Page Number: ");
    scanf("%d", &pageNumber);
    printf("Enter Offset: ");
    scanf("%d", &offset);
    if (pageNumber < 0 || pageNumber >= pages) {
        printf("\nInvalid Page Number!\n");
    } else if (offset < 0 || offset >= pageSize) {
        printf("\nInvalid Offset!\n");
    }
}
```

```

    } else{
        frameNumber = pageTable[pageNumber];
        physicalAddress = (frameNumber * pageSize) + offset;
        printf("\nPage Number    = %d", pageNumber);
        printf("\nFrame Number    = %d", frameNumber);
        printf("\nOffset          = %d", offset);
        printf("\nPhysical Address = %d\n", physicalAddress);
    }
    return 0;
}

```

OUTPUT:

Programiz
C Online Compiler

```

main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int pageTable[100];
6     int pages, pageSize;
7     int pageNumber, offset;
8     int frameNumber, physicalAddress;
9
10    printf("Enter number of pages: ");
11    scanf("%d", &pages);
12
13    printf("Enter page size: ");
14    scanf("%d", &pageSize);
15
16    printf("\nEnter Page Table Entries:\n");
17
18    for (int i = 0; i < pages; i++)
19    {
20        printf("Page %d -> Frame: ", i);
21        scanf("%d", &pageTable[i]);
22    }
23
24    printf("\nPage Table:\n");
25    printf("-----\n");
26    printf("Page Number    Frame Number\n");
27    printf("-----\n");
28
29    for (int i = 0; i < pages; i++)
30    {
31        printf("    %d          %d\n", i,
32              pageTable[i]);
33    }
34
35    printf("\nEnter Logical Address:\n");
36
37    printf("Enter Page Number: ");
38    scanf("%d", &pageNumber);
39
40    printf("Enter Offset: ");
41    scanf("%d", &offset);

```

Output

```

Enter number of pages: 4
Enter page size: 100

Enter Page Table Entries:
Page 0 -> Frame: 3
Page 1 -> Frame: 5
Page 2 -> Frame: 1
Page 3 -> Frame: 4

Page Table:
-----
Page Number    Frame Number
-----
0              3
1              5
2              1
3              4

Enter Logical Address:
Enter Page Number: 2
Enter Offset: 25

Page Number     = 2
Frame Number    = 1
Offset          = 25
Physical Address = 125

=== Code Execution Successful ===

```

4. Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.

PSEUDO CODE:

```
START
Read memory access time
Read TLB access time
Read TLB hit ratio
Hit Ratio = Hit Ratio / 100
Miss Ratio = 1 - Hit Ratio
EAT without TLB = 2 × Memory Access Time
EAT with TLB =
    Hit Ratio × (TLB Time + Memory Time)
    + Miss Ratio × (TLB Time + 2 × Memory Time)
Display EAT without TLB
Display EAT with TLB
IF EAT with TLB < EAT without TLB
    Display "TLB improves memory access time"
ELSE
    Display "TLB does not improve memory access time"
STOP
```

CODE:

```
#include <stdio.h>

int main()
{
    int memoryTime, tlbTime, hitRatio;
    float hit, eatWithTLB, eatWithoutTLB;
    printf("Enter Memory Access Time (in ns): ");
    scanf("%d", &memoryTime);
    printf("Enter TLB Access Time (in ns): ");
    scanf("%d", &tlbTime);
```

```

printf("Enter TLB Hit Ratio (in %%): ");
scanf("%d", &hitRatio);
hit = hitRatio / 100.0;
// Without TLB
eatWithoutTLB = 2 * memoryTime;
// With TLB
// Hit: TLB access + one memory access
// Miss: TLB access + two memory accesses
eatWithTLB = hit * (tlbTime + memoryTime)
              + (1 - hit) * (tlbTime + 2 * memoryTime);
printf("\n----- TLB Performance -----\n");
printf("Memory Access Time    = %d ns\n", memoryTime);
printf("TLB Access Time       = %d ns\n", tlbTime);
printf("TLB Hit Ratio          = %d%%\n", hitRatio);
printf("\nEffective Access Time WITHOUT TLB = %.2f ns\n",
       eatWithoutTLB);
printf("Effective Access Time WITH TLB   = %.2f ns\n",
       eatWithTLB);
if (eatWithTLB < eatWithoutTLB)
    printf("\nTLB improves memory access time.\n");
else
    printf("\nTLB does not improve memory access time.\n");
return 0;

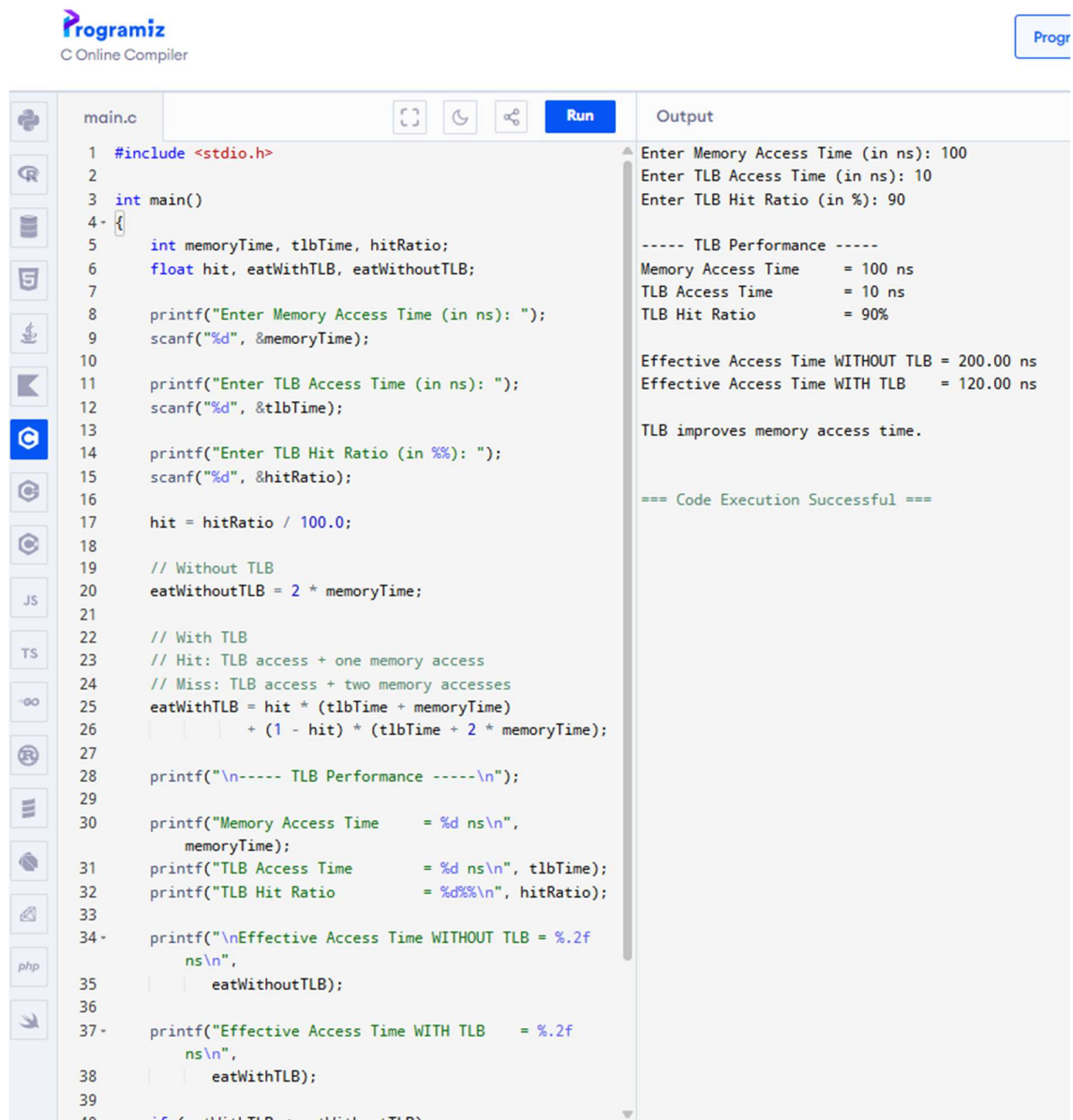
```

```

}

```

OUTPUT:



The screenshot displays the Programiz C Online Compiler interface. The left sidebar contains icons for various programming languages, with C selected. The main editor shows a C program named 'main.c' that simulates TLB performance. The code prompts the user for memory access time, TLB access time, and TLB hit ratio. It then calculates the effective access time with and without the TLB. The right pane shows the output of the program, which matches the input values and calculations shown in the code.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int memoryTime, tlbTime, hitRatio;
6     float hit, eatWithTLB, eatWithoutTLB;
7
8     printf("Enter Memory Access Time (in ns): ");
9     scanf("%d", &memoryTime);
10
11    printf("Enter TLB Access Time (in ns): ");
12    scanf("%d", &tlbTime);
13
14    printf("Enter TLB Hit Ratio (in %): ");
15    scanf("%d", &hitRatio);
16
17    hit = hitRatio / 100.0;
18
19    // Without TLB
20    eatWithoutTLB = 2 * memoryTime;
21
22    // With TLB
23    // Hit: TLB access + one memory access
24    // Miss: TLB access + two memory accesses
25    eatWithTLB = hit * (tlbTime + memoryTime)
26    + (1 - hit) * (tlbTime + 2 * memoryTime);
27
28    printf("\n----- TLB Performance -----");
29
30    printf("Memory Access Time      = %d ns\n",
31           memoryTime);
32    printf("TLB Access Time           = %d ns\n", tlbTime);
33    printf("TLB Hit Ratio                = %d%%\n", hitRatio);
34
35    printf("\nEffective Access Time WITHOUT TLB = %.2f\n",
36           eatWithoutTLB);
37
38    printf("Effective Access Time WITH TLB   = %.2f\n",
39           eatWithTLB);
40    printf("Difference = %.2f\n", eatWithoutTLB - eatWithTLB);
41 }
```

Output

```
Enter Memory Access Time (in ns): 100
Enter TLB Access Time (in ns): 10
Enter TLB Hit Ratio (in %): 90

----- TLB Performance -----
Memory Access Time      = 100 ns
TLB Access Time         = 10 ns
TLB Hit Ratio           = 90%

Effective Access Time WITHOUT TLB = 200.00 ns
Effective Access Time WITH TLB   = 120.00 ns

TLB improves memory access time.

=== Code Execution Successful ===
```

5. Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

PSEUDO CODE:

```
START
Read Base Register
Read Limit Register
Read Logical Address
IF Logical Address < 0 OR Logical Address >= Limit THEN
    Display "ACCESS DENIED"
    Display "Invalid Logical Address"
ELSE
    Physical Address = Base + Logical Address
    Display "ACCESS ALLOWED"
    Display Physical Address
END IF
STOP
```

CODE:

```
#include <stdio.h>
int main()
{
    int base, limit, logicalAddress, physicalAddress;
    printf("Enter Base Register: ");
    scanf("%d", &base);
    printf("Enter Limit Register: ");
    scanf("%d", &limit);
    printf("Enter Logical Address: ");
    scanf("%d", &logicalAddress);
    // Check whether the logical address is within the limit
    if (logicalAddress < 0 || logicalAddress >= limit)
```

```

    {
        printf("\nACCESS DENIED!\n");
        printf("Invalid Logical Address.\n");
        printf("Address is outside the protected memory range.\n");
    }else {
        physicalAddress = base + logicalAddress;
        printf("\nACCESS ALLOWED!\n");
        printf("Logical Address = %d\n", logicalAddress);
        printf("Base Register = %d\n", base);
        printf("Limit Register = %d\n", limit);
        printf("Physical Address = %d\n", physicalAddress);
    }
    return 0;
}

```

OUTPUT:

VALID INPUT:

Programiz
C Online Compiler

main.c	Output
<pre> 1 #include <stdio.h> 2 3 int main() 4 { 5 int base, limit, logicalAddress, physicalAddress; 6 7 printf("Enter Base Register: "); 8 scanf("%d", &base); 9 10 printf("Enter Limit Register: "); 11 scanf("%d", &limit); 12 13 printf("Enter Logical Address: "); 14 scanf("%d", &logicalAddress); 15 16 // Check whether the logical address is within the 17 // limit 18 if (logicalAddress < 0 logicalAddress >= limit) 19 { 20 printf("\nACCESS DENIED!\n"); 21 printf("Invalid Logical Address.\n"); 22 printf("Address is outside the protected memory 23 range.\n"); 24 } 25 else 26 { 27 physicalAddress = base + logicalAddress; 28 29 printf("\nACCESS ALLOWED!\n"); 30 printf("Logical Address = %d\n", logicalAddress); 31 printf("Base Register = %d\n", base); 32 printf("Limit Register = %d\n", limit); 33 printf("Physical Address = %d\n", physicalAddress); 34 } 35 return 0; 36 } </pre>	<pre> Enter Base Register: 1000 Enter Limit Register: 500 Enter Logical Address: 250 ACCESS ALLOWED! Logical Address = 250 Base Register = 1000 Limit Register = 500 Physical Address = 1250 === Code Execution Successful === </pre>

INVALID INPUT:


C Online Compiler

Programiz

main.c

1 #include <stdio.h>

2

3 int main()

4 {

5 int base, limit, logicalAddress, physicalAddress;

6

7 printf("Enter Base Register: ");

8 scanf("%d", &base);

9

10 printf("Enter Limit Register: ");

11 scanf("%d", &limit);

12

13 printf("Enter Logical Address: ");

14 scanf("%d", &logicalAddress);

15

16 // Check whether the logical address is within the limit

17 if (logicalAddress < 0 || logicalAddress >= limit)

18 {

19 printf("\nACCESS DENIED!\n");

20 printf("Invalid Logical Address.\n");

21 printf("Address is outside the protected memory range.\n");

22 }

23 else{

24 physicalAddress = base + logicalAddress;

25

26 printf("\nACCESS ALLOWED!\n");

27 printf("Logical Address = %d\n", logicalAddress);

28 printf("Base Register = %d\n", base);

29 printf("Limit Register = %d\n", limit);

30 printf("Physical Address = %d\n", physicalAddress);

31 }

32 }

33 return 0;

34 }

Run

Output

Enter Base Register: 1000
Enter Limit Register: 500
Enter Logical Address: 600

ACCESS DENIED!
Invalid Logical Address.
Address is outside the protected memory range.

=== Code Execution Successful ===